

Compiler

Abstract syntax, semantics

Jyun-Ao Lin

iFIRST & CSIE, NTUT

A large part of this course is based on the Compilation Course of J.-C. Filliâtre at ENS Ulm.

Overview of the course

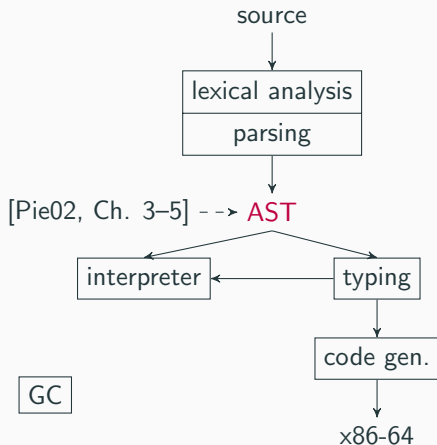


Table of contents

1. ABSTRACT SYNTAX

2. OPERATIONAL SEMANTICS

mini-ML

big-steps semantics

3. INTERPRETER

4. SMALL-STEPS SEMANTICS

equivalence between big-steps and small-steps semantics

Abstract syntax

meaning

how to define the meaning of programs?

meaning

how to define the meaning of programs?

most of the time, we are satisfied with an informal specification, in natural language (e.g. ISO norm, standard, reference book)

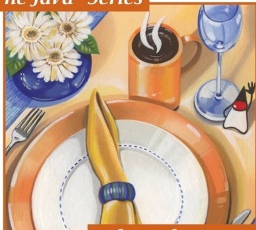
yet it is imprecise, sometimes even ambiguous

informal semantics

James Gosling • Bill Joy • Guy Steele • Gilad Bracha ↗

The Java™ Language Specification, Third Edition

The Java™ Series



...from the Source™

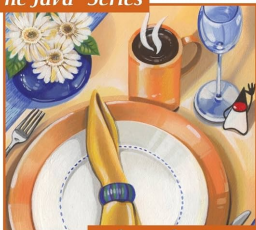
The Java programming language guarantees that the operands of operators appear to be evaluated in a specific evaluation order, namely, from left to right.

informal semantics

James Gosling • Bill Joy • Guy Steele • Gilad Bracha ↕

The Java™ Language Specification, Third Edition

The Java™ Series



...from the Source™

The Java programming language guarantees that the operands of operators appear to be evaluated in a specific evaluation order, namely, from left to right.

it is recommended that code not rely crucially on this specification.

formal semantics

the **formal semantics** gives a mathematical characterization of the computations defined by a program

useful to make tools (interpreters, compilers, etc.)

necessary to reason about programs

raise another question

what is a program?

raise another question

what is a program?

as a syntactic object (sequence of characters),
it is too complex to apprehend

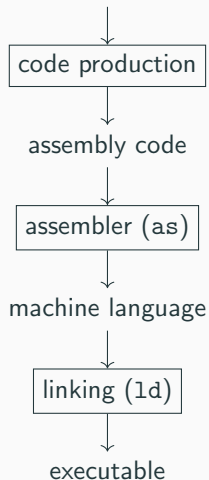
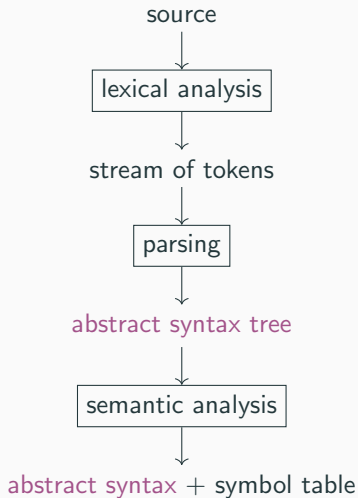
raise another question

what is a program?

as a syntactic object (sequence of characters),
it is too complex to apprehend

that's why we switch to **abstract syntax**.

abstract syntax



abstract syntax

the text

$2*(x+1)$

abstract syntax

the text

```
2*(x+1)
```

and

```
(2 * ((x)+1))
```

abstract syntax

the text

```
2*(x+1)
```

and

```
(2 * ((x)+1))
```

and

```
2 * (* I double it *) ( x + 1 )
```

abstract syntax

the text

```
2*(x+1)
```

and

```
(2 * ((x)+1))
```

and

```
2 * (* I double it *) ( x + 1 )
```

all map to the same **abstract syntax tree**



notation

we define the abstract syntax using a (BNF) **grammar**

e	$::=$	c	constant
	$ $	x	variable
	$ $	$e + e$	addition
	$ $	$e \times e$	multiplication
	$ $	$\dots$	

it reads “an expression, denoted by e , is

- either a constant c ,
- either a variable x ,
- either the addition of two expressions,
- etc.”

here $c, x, e, \dots$ etc are called **meta-variables**, that is, c denotes a whatever constant, ...etc

notation

the notation $e_1 + e_2$ of the abstract syntax borrows the symbol of the concrete syntax

but we could have picked something else, e.g. $add(e_1, e_2)$, $+(e_1, e_2)$, etc.

abstract syntax in Java

we use classes to build abstract syntax trees, as follows:

```
enum Binop { Add, Mul, ... }

abstract class Expr {}
class Cte extends Expr { int n; }
class Var extends Expr { string x; }
class Bin extends Expr { Binop op; Expr e1, e2; }
...
```

(constructors are omitted)

expression $2 * (x + 1)$ is then represented as

```
new Bin(Mul, new Cte(2), new Bin(Add, new Var("x"), new Cte(1)))
```

abstract syntax in OCaml

we use algebraic data types to abstract syntax trees, as follows:

```
type binop = Add | Mul | ...

type expr =
  | Cte of int
  | Var of string
  | Bin of binop * expr * expr
  | ...
```

abstract syntax in OCaml

we use algebraic data types to abstract syntax trees, as follows:

```
type binop = Add | Mul | ...

type expr =
  | Cte of int
  | Var of string
  | Bin of binop * expr * expr
  | ...
```

expression $2 * (x + 1)$ is then represented as

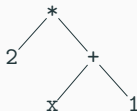
```
Bin (Mul, Cte 2, Bin (Add, Var "x", Cte 1))
```

parentheses

there is no construct for parentheses in abstract syntax

in **concrete** syntax $2 * (x + 1)$,

parentheses are used to build this tree



rather than this one



(the lecture on parsing will explain how)

syntactic sugar

we call **syntactic sugar** a construct of concrete syntax that does not exist in abstract syntax

it is thus translated in terms of other constructs of abstract syntax
(typically during parsing)

Example

syntactic sugar

we call **syntactic sugar** a construct of concrete syntax that does not exist in abstract syntax

it is thus translated in terms of other constructs of abstract syntax (typically during parsing)

Example

- in C, expression `a[i]` is syntactic sugar for `*(a+i)`.

syntactic sugar

we call **syntactic sugar** a construct of concrete syntax that does not exist in abstract syntax

it is thus translated in terms of other constructs of abstract syntax (typically during parsing)

Example

- in C, expression `a[i]` is syntactic sugar for `*(a+i)`.
- in OCAML, expression `[e1; e2; ...; en]` is sugar for `e1 :: e2 :: ... :: en :: []`.

operational semantics

semantics

formal semantics is defined over abstract syntax

there are many approaches

- axiomatic semantics
- denotational semantics
- semantics by translation
- operational semantics

axiomatic semantics

axiomatic semantics is also called (Floyd-)Hoare logic

(Robert Floyd, *Assigning meanings to programs*, 1967

Tony Hoare, *An axiomatic basis for computer programming*, 1969)

it defines programs by means of their properties; we introduce a triple

$$\{P\} i \{Q\}$$

meaning "if formula P holds before the execution of statement i , then formula Q holds after the execution".

axiomatic semantics

axiomatic semantics is also called (Floyd-)Hoare logic

(Robert Floyd, *Assigning meanings to programs*, 1967

Tony Hoare, *An axiomatic basis for computer programming*, 1969)

it defines programs by means of their properties; we introduce a triple

$$\{P\} i \{Q\}$$

meaning "if formula P holds before the execution of statement i , then formula Q holds after the execution".

Example

- example

$$\{x \geq 0\} x := x + 1 \{x > 0\}$$

- example of rule:

$$\{P[x \leftarrow E]\} x := E \{P(x)\}$$

denotational Semantics

denotational semantics maps each program expression e to its denotation $\llbracket e \rrbracket$, a mathematical object that represents the computation denoted by e .

denotational Semantics

denotational semantics maps each program expression e to its denotation $\llbracket e \rrbracket$, a mathematical object that represents the computation denoted by e .

Example

arithmetic expressions with a single variable x

$$e ::= x \mid n \mid e + e \mid e * e \mid \dots$$

the denotation is a function that maps the value of x to the value of the expression

$$\llbracket x \rrbracket = x \mapsto x$$

$$\llbracket n \rrbracket = x \mapsto n$$

$$\llbracket e_1 + e_2 \rrbracket = x \mapsto \llbracket e_1 \rrbracket(x) + \llbracket e_2 \rrbracket(x)$$

$$\llbracket e_1 * e_2 \rrbracket = x \mapsto \llbracket e_1 \rrbracket(x) \times \llbracket e_2 \rrbracket(x)$$

semantics by translation

(also called Strachey semantics)

we can define the semantics of a language by means of its translation to another language for which the semantics is already defined.

example of semantics by translation

an esoteric language whose syntax consists of 8 characters and whose semantics is defined by translation to the C language

command	translation to C
(prelude)	<code>char array[3000] = 0</code> <code>char *ptr = array;</code>
>	<code>++ptr;</code>
<	<code>--ptr;</code>
+	<code>++*ptr;</code>
-	<code>--*ptr;</code>
.	<code>putchar(*ptr);</code>
,	<code>*ptr = getchar();</code>
[	<code>while (*ptr) {</code>
]	<code>}</code>

operational semantics

operational semantics describes the sequence of elementary computations from the expression to its outcome (its value)

it operates directly over abstract syntax

operational semantics

operational semantics describes the sequence of elementary computations from the expression to its outcome (its value)

it operates directly over abstract syntax

two kinds of operational semantics

operational semantics

operational semantics describes the sequence of elementary computations from the expression to its outcome (its value)

it operates directly over abstract syntax

two kinds of operational semantics

- "natural semantics" or "big-steps"

$$e \twoheadrightarrow v$$

operational semantics

operational semantics describes the sequence of elementary computations from the expression to its outcome (its value)

it operates directly over abstract syntax

two kinds of operational semantics

- "natural semantics" or "big-steps"

$$e \twoheadrightarrow v$$

- "reduction semantics" or "small-steps"

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

mini-ML

mini-ML

let us illustrate big-steps operational semantics on the language **mini-ML**

$e ::=$	x	variable
	c	constant $(1, 2, \dots, \text{true}, \dots)$
	op	primitive operator $(+, \times, \text{fst}, \dots)$
	$\text{fun } x \rightarrow e$	function
	$e\ e$	application
	(e, e)	pair
	$\text{let } x = e \text{ in } e$	local declaration

example

```
let compose = fun f -> fun g -> fun x -> f (g x) in
let plus = fun x -> fun y -> + (x,y) in
compose (plus 2) (plus 4) 36
```

example

```
let compose = fun f -> fun g -> fun x -> f (g x) in
let plus = fun x -> fun y -> + (x,y) in
compose (plus 2) (plus 4) 36
```

```
let distr_pair = fun f -> fun p -> (f (fst p), f (snd p)) in
let p = distr_pair (fun x -> x) (40,2) in
+ (fst p, snd p)
```

another construct : conditional

the conditional can be defined as

$$\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \stackrel{\text{def}}{=} \text{opif}(e_1, ((\text{fun } _ \rightarrow e_2), (\text{fun } _ \rightarrow e_3)))$$

where *opif* is a primitive

the branches are **frozen** using function

another construct : recursive

similarly, the recursive can be defined as

$$\text{rec } f \ x = e \quad \stackrel{\text{def}}{=} \quad \text{opfix}(\text{fun } f \rightarrow \text{fun } x \rightarrow e)$$

where *opfix* is an operator of fixed point, satisfying

$$\text{opfix } f = f \ (\text{opfix } f)$$

Example

$$\begin{aligned} &\text{opfix } (\text{fun } \text{fact} \rightarrow \text{fun } n \rightarrow \text{if } n = 0 \text{ then } 1 \\ &\quad \text{else } \times (n, \text{fact}(-(n, 1)))) \end{aligned}$$

expressiveness

MINI-ML is Turing-complete!

big-steps semantics of mini-ML

big-steps operational semantics of mini-ML

we seek to define a **relation** between some expression e and a **value** v

$$e \twoheadrightarrow v$$

big-steps operational semantics of mini-ML

we seek to define a **relation** between some expression e and a **value** v

$$e \twoheadrightarrow v$$

here, values are defined as

$v ::=$	c	constant
	$ \quad op$	not applied primitive
	$ \quad \text{fun } x \rightarrow e$	function
	$ \quad (v, v)$	pair

big-steps operational semantics of mini-ML

we seek to define a **relation** between some expression e and a **value** v

$$e \twoheadrightarrow v$$

here, values are defined as

$v ::=$	c	constant
	$ \quad op$	not applied primitive
	$ \quad \text{fun } x \rightarrow e$	function
	$ \quad (v, v)$	pair

to define $e \twoheadrightarrow v$, one needs the notions of **inference rules** and of **substitution**.

inference rules

a relation may be defined as the **smallest relation** satisfying
a set of rules with no premises (axioms) written as

$$\overline{P}$$

and a set of rules with premises written as

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{P}$$

inference rules

a relation may be defined as the **smallest relation** satisfying a set of rules with no premises (axioms) written as

$$\overline{P}$$

and a set of rules with premises written as

$$\frac{P_1 \quad P_2 \quad \dots \quad P_n}{P}$$

these are called **inference rules**

example

we can define the relation $\text{Even}(n)$ with two rules

$$\frac{}{\text{Even}(0)} \quad \text{and} \quad \frac{\text{Even}(n)}{\text{Even}(n+2)}$$

that reads as follows

on the one hand $\text{Even}(0)$

on the other hand $\forall n. \text{Even}(n) \implies \text{Even}(n+2)$

the smallest relation satisfying these two properties coincide with the property " n is an even natural number":

- even natural numbers are included, by induction
- if odd numbers were included, we could remove the smallest

derivation tree

a **derivation** is a tree whose internal nodes are rules with premises and whose leaves are axioms.

derivation tree

a **derivation** is a tree whose internal nodes are rules with premises and whose leaves are axioms.

Example

$$\frac{\frac{\frac{}{\text{Even}(0)}}{\text{Even}(2)}}{\text{Even}(4)}$$

derivation tree

a **derivation** is a tree whose internal nodes are rules with premises and whose leaves are axioms.

Example

$$\frac{\frac{\frac{}{\text{Even}(0)}}{\text{Even}(2)}}{\text{Even}(4)}$$

the set of derivation trees characterizes the smallest relation satisfying the inference rules

free variables

Defn. (free variables)

The set of **free variables** of an expression e , denoted by $fv(e)$, is defined recursively over e as follows:

$$\begin{aligned}fv(x) &= \{x\} \\fv(c) &= \emptyset \\fv(op) &= \emptyset \\fv(\text{fun } x \rightarrow e) &= fv(e) \setminus \{x\} \\fv(e_1 \ e_2) &= fv(e_1) \cup fv(e_2) \\fv((e_1, e_2)) &= fv(e_1) \cup fv(e_2) \\fv(\text{let } x = e_1 \text{ in } e_2) &= fv(e_1) \cup (fv(e_2) \setminus \{x\})\end{aligned}$$

an expression without free variables is called **closed**



examples

Example

- $fv(\text{let } x = +(20, 1) \text{ in } ((\text{fun } y \rightarrow +(y, y)) x)) =$

examples

Example

- $fv(\text{let } x = +(20, 1) \text{ in } ((\text{fun } y \rightarrow +(y, y)) x)) = \emptyset$

examples

Example

- $fv(\text{let } x = +(20, 1) \text{ in } ((\text{fun } y \rightarrow +(y, y)) x)) = \emptyset$
- $fv(\text{let } x = z \text{ in } (\text{fun } y \rightarrow (x y) t)) =$

examples

Example

- $fv(\text{let } x = +(20, 1) \text{ in } ((\text{fun } y \rightarrow +(y, y)) x)) = \emptyset$
- $fv(\text{let } x = z \text{ in } (\text{fun } y \rightarrow (x y) t)) = \{z, t\}$

substitution

Defn. (substitution)

Let e be an expression, x a variable and v a value, we denote by $e[x \leftarrow v]$ the **substitution** of any free occurrence of x in e by v and is defined by

$$x[x \leftarrow v] = v$$

$$y[x \leftarrow v] = y \quad \text{if } y \neq x$$

$$c[x \leftarrow v] = c$$

$$op[x \leftarrow v] = op$$

$$(\text{fun } x \rightarrow e)[x \leftarrow v] = \text{fun } x \rightarrow e$$

$$(\text{fun } y \rightarrow e)[x \leftarrow v] = \text{fun } y \rightarrow e[x \leftarrow v] \quad \text{if } y \neq x$$

$$(e_1 e_2)[x \leftarrow v] = (e_1[x \leftarrow v] e_2[x \leftarrow v])$$

$$(e_1, e_2)[x \leftarrow v] = (e_1[x \leftarrow v], e_2[x \leftarrow v])$$

$$(\text{let } x = e_1 \text{ in } e_2)[x \leftarrow v] = (\text{let } x = e_1[x \leftarrow v] \text{ in } e_2)$$

$$(\text{let } y = e_1 \text{ in } e_2)[x \leftarrow v] = (\text{let } y = e_1[x \leftarrow v] \text{ in } e_2[x \leftarrow v]) \quad \text{if } y \neq x$$



examples

Example

- $((\text{fun } x \rightarrow +(x, x)) \text{ } x)[x \leftarrow 21] =$

examples

Example

- $((\text{fun } x \rightarrow +(x, x)) \text{ } x)[x \leftarrow 21] = (\text{fun } x \rightarrow +(x, x)) \text{ } 21$

examples

Example

- $((\text{fun } x \rightarrow +(x, x)) \text{ } x)[x \leftarrow 21] = (\text{fun } x \rightarrow +(x, x)) \text{ } 21$
- $(+(\text{ } x, \text{let } x = 17 \text{ in } x))[x \leftarrow 3] =$

examples

Example

- $((\text{fun } x \rightarrow +(x, x)) \text{ } x)[x \leftarrow 21] = (\text{fun } x \rightarrow +(x, x)) \text{ } 21$
- $(+(\text{ } x, \text{let } x = 17 \text{ in } x))[x \leftarrow 3] = +(3, \text{let } x = 17 \text{ in } x)$

examples

Example

- $((\text{fun } x \rightarrow +(x, x)) \text{ } x)[x \leftarrow 21] = (\text{fun } x \rightarrow +(x, x)) 21$
- $(+(\text{ } x, \text{let } x = 17 \text{ in } x))[x \leftarrow 3] = +(3, \text{let } x = 17 \text{ in } x)$
- $(\text{fun } y \rightarrow (y \ y))[y \leftarrow 17] =$

examples

Example

- $((\text{fun } x \rightarrow +(x, x)) \text{ } x)[x \leftarrow 21] = (\text{fun } x \rightarrow +(x, x)) \text{ } 21$
- $(+(\text{ } x, \text{let } x = 17 \text{ in } x))[x \leftarrow 3] = +(3, \text{let } x = 17 \text{ in } x)$
- $(\text{fun } y \rightarrow (y \text{ } y))[y \leftarrow 17] = \text{fun } y \rightarrow y \text{ } y$

natural semantics of mini-ML

$$\begin{array}{c}
 \frac{}{c \twoheadrightarrow c} \qquad \frac{}{op \twoheadrightarrow op} \qquad \frac{}{(\text{fun } x \rightarrow e) \twoheadrightarrow (\text{fun } x \rightarrow e)} \\
 \\
 \frac{e_1 \twoheadrightarrow v_1 \quad e_2 \twoheadrightarrow v_2}{(e_1, e_2) \twoheadrightarrow (v_1, v_2)} \qquad \frac{e_1 \twoheadrightarrow v_1 \quad e_2[x \leftarrow v_1] \twoheadrightarrow v}{\text{let } x = e_1 \text{ in } e_2 \twoheadrightarrow v} \\
 \\
 \frac{e_1 \twoheadrightarrow (\text{fun } x \rightarrow e) \quad e_2 \twoheadrightarrow v_2 \quad e[x \leftarrow v_2] \twoheadrightarrow v}{e_1 \ e_2 \twoheadrightarrow v}
 \end{array}$$

note: we have chosen a strategy of **call by value**. i.e. the argument is completely evaluated before the call.

primitive

we have to add the rules for the primitives, for example,

$$\frac{e_1 \twoheadrightarrow + \quad e_2 \twoheadrightarrow (n_1, n_2) \quad n = n_1 + n_2}{e_1 \ e_2 \twoheadrightarrow n}$$

$$\frac{e_1 \twoheadrightarrow \text{opif} \quad e_2 \twoheadrightarrow (\text{true}, ((\text{fun } _ \rightarrow e_3), (\text{fun } _ \rightarrow e_4)))) \quad e_3 \twoheadrightarrow v}{e_1 \ e_2 \twoheadrightarrow v}$$

$$\frac{e_1 \twoheadrightarrow \text{opfix} \quad e_2 \twoheadrightarrow (\text{fun } f \rightarrow e) \quad e[f \leftarrow \text{opfix}(\text{fun } f \rightarrow e)] \twoheadrightarrow v}{e_1 \ e_2 \twoheadrightarrow v}$$

$$\frac{e_1 \twoheadrightarrow \text{fst} \quad e_2 \twoheadrightarrow (v_1, v_2)}{e_1 \ e_2 \twoheadrightarrow v_1}$$

example of derivation

$$\begin{array}{c}
 \frac{}{+ \twoheadrightarrow +} \quad \frac{\frac{}{20 \twoheadrightarrow 20} \quad \frac{}{1 \twoheadrightarrow 1}}{(20, 1) \twoheadrightarrow (20, 1)} \quad \frac{}{\text{fun } \dots \twoheadrightarrow \text{fun } \dots} \quad \frac{}{21 \twoheadrightarrow 21} \quad \frac{\vdots}{+(21, 21) \twoheadrightarrow 42} \\
 \hline
 \frac{+ \twoheadrightarrow + \quad (20, 1) \twoheadrightarrow (20, 1) \quad \text{fun } \dots \twoheadrightarrow \text{fun } \dots \quad 21 \twoheadrightarrow 21 \quad +(21, 21) \twoheadrightarrow 42}{+(20, 1) \twoheadrightarrow 21 \quad (\text{fun } y \twoheadrightarrow +(y, y)) 21 \twoheadrightarrow 42} \\
 \hline
 \text{let } x = +(20, 1) \text{ in } (\text{fun } y \twoheadrightarrow +(y, y)) x \twoheadrightarrow 42
 \end{array}$$

exercise

give the derivation of

$$(opfix\ F)\ 2$$

where F is defined as

$$\text{fun } fact \rightarrow \text{fun } n \rightarrow \text{if } n = 0 \text{ then } 1 \text{ else } \times (n, fact(-(n, 1)))$$

expression without value

there are expressions e for which there is **no value** v such that $e \rightarrow v$

Example

- $e = 1\ 2$
- $e = (\text{fun } x \rightarrow x\ x)\ (\text{fun } x \rightarrow x\ x)$

induction over the derivation

to establish a property of a relation defined by a set of inference rules, we can reason **by induction** on the derivation

that is, by **structural induction**, i.e. we can apply the induction hypothesis to any subderivation.

(equivalently, we may say that we reason by induction on the height of derivation)

in practice, we proceed by induction on the derivation and **by case** on the last rule used.

properties of the natural semantics of mini-ML

Prop.

If $e \twoheadrightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \twoheadrightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) & e_2 \twoheadrightarrow v_2 & e_3[x \leftarrow v_2] \twoheadrightarrow v \\
 \hline
 e_1 \ e_2 \twoheadrightarrow v
 \end{array}$$

properties of the natural semantics of mini-ML

Prop.

If $e \twoheadrightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \twoheadrightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) & e_2 \twoheadrightarrow v_2 & e_3[x \leftarrow v_2] \twoheadrightarrow v \\
 \hline
 e_1 \ e_2 \twoheadrightarrow v
 \end{array}$$

If $e = e_1 \ e_2$ is close, then e_1 and e_2 are close.

properties of the natural semantics of mini-ML

Prop.

If $e \twoheadrightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \twoheadrightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) & e_2 \twoheadrightarrow v_2 & e_3[x \leftarrow v_2] \twoheadrightarrow v \\
 \hline
 e_1 \ e_2 \twoheadrightarrow v
 \end{array}$$

If $e = e_1 \ e_2$ is close, then e_1 and e_2 are close. By induction hypothesis, we have

- $\text{fun } x \rightarrow e_3$ is a close value.
- v_2 is a close value.

properties of the natural semantics of mini-ML

Prop.

If $e \rightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \rightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \rightarrow (\text{fun } x \rightarrow e_3) & e_2 \rightarrow v_2 & e_3[x \leftarrow v_2] \rightarrow v \\
 \hline
 e_1 \ e_2 \rightarrow v
 \end{array}$$

If $e = e_1 \ e_2$ is close, then e_1 and e_2 are close. By induction hypothesis, we have

- $\text{fun } x \rightarrow e_3$ is a close value.
- v_2 is a close value.

Since $\text{fun } x \rightarrow e_3$ is close, the only free variable in e_3 is x .

properties of the natural semantics of mini-ML

Prop.

If $e \twoheadrightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \twoheadrightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) & e_2 \twoheadrightarrow v_2 & e_3[x \leftarrow v_2] \twoheadrightarrow v \\
 \hline
 e_1 \ e_2 \twoheadrightarrow v
 \end{array}$$

If $e = e_1 \ e_2$ is close, then e_1 and e_2 are close. By induction hypothesis, we have

- $\text{fun } x \rightarrow e_3$ is a close value.
- v_2 is a close value.

Since $\text{fun } x \rightarrow e_3$ is close, the only free variable in e_3 is x . Thus $e_3[x \leftarrow v_2]$ is close.

properties of the natural semantics of mini-ML

Prop.

If $e \twoheadrightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \twoheadrightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) & e_2 \twoheadrightarrow v_2 & e_3[x \leftarrow v_2] \twoheadrightarrow v \\
 \hline
 e_1 \ e_2 \twoheadrightarrow v
 \end{array}$$

If $e = e_1 \ e_2$ is close, then e_1 and e_2 are close. By induction hypothesis, we have

- $\text{fun } x \rightarrow e_3$ is a close value.
- v_2 is a close value.

Since $\text{fun } x \rightarrow e_3$ is close, the only free variable in e_3 is x . Thus $e_3[x \leftarrow v_2]$ is close.

By induction hypothesis, v is close.

properties of the natural semantics of mini-ML

Prop.

If $e \twoheadrightarrow v$ then v is a value.

Moreover, if e is close, then so is v .



Proof.

By induction on the derivation $e \twoheadrightarrow v$. Consider the case of an application,

$$\begin{array}{ccc}
 (D_1) & (D_2) & (D_3) \\
 \vdots & \vdots & \vdots \\
 e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) & e_2 \twoheadrightarrow v_2 & e_3[x \leftarrow v_2] \twoheadrightarrow v \\
 \hline
 e_1 \ e_2 \twoheadrightarrow v
 \end{array}$$

If $e = e_1 \ e_2$ is close, then e_1 and e_2 are close. By induction hypothesis, we have

- $\text{fun } x \rightarrow e_3$ is a close value.
- v_2 is a close value.

Since $\text{fun } x \rightarrow e_3$ is close, the only free variable in e_3 is x . Thus $e_3[x \leftarrow v_2]$ is close. By induction hypothesis, v is close. The other cases are left as exercises.



properties of natural semantics of mini-ML

Prop. (evaluation is deterministic)

If $e \twoheadrightarrow v$ and $e \twoheadrightarrow v'$ then $v = v'$.



Proof.

We proceed by induction on the derivations of $e \twoheadrightarrow v$ and $e \twoheadrightarrow v'$. Let us examine the case of a pair $e = (e_1, e_2)$.

$$\begin{array}{c}
 \begin{array}{cc}
 (D_1) & (D_2) \\
 \vdots & \vdots \\
 e_1 \twoheadrightarrow v_1 & e_2 \twoheadrightarrow v_2
 \end{array} \\
 \hline
 (e_1, e_2) \twoheadrightarrow (v_1, v_2)
 \end{array}
 \qquad
 \begin{array}{c}
 \begin{array}{cc}
 (D'_1) & (D'_2) \\
 \vdots & \vdots \\
 e_1 \twoheadrightarrow v'_1 & e_2 \twoheadrightarrow v'_2
 \end{array} \\
 \hline
 (e_1, e_2) \twoheadrightarrow (v'_1, v'_2)
 \end{array}$$

By induction hypothesis, we have $v_1 = v'_1$ and $v_2 = v'_2$. Hence $v = (v_1, v_2) = (v'_1, v'_2) = v'$.

The other cases are left as exercises.



determinism

Remark.

an evaluation relation is not necessarily deterministic



determinism

Remark.

an evaluation relation is not necessarily deterministic



Example

we add a primitive *random* and the rule

$$\frac{e_1 \twoheadrightarrow \text{random} \quad e_2 \twoheadrightarrow n_1 \quad 0 \leq n < n_1}{e_1 \ e_2 \twoheadrightarrow n}$$

then we can have *random* 2 $\twoheadrightarrow$ 0 but also *random* 2 $\twoheadrightarrow$ 1.

remarks on *opif* and *opfix*

Remark.

- recall that we defined the conditional as

$$\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \stackrel{\text{def}}{=} \text{opif}(e_1, ((\text{fun } _ \rightarrow e_2), (\text{fun } _ \rightarrow e_3)))$$

remarks on *opif* and *opfix*

Remark.

- recall that we defined the conditional as

$$\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \stackrel{\text{def}}{=} \text{opif}(e_1, ((\text{fun } _ \rightarrow e_2), (\text{fun } _ \rightarrow e_3)))$$

we can define Boolean constants and the primitive operator *opif* as

$$\text{true} \stackrel{\text{def}}{=} \text{fun } x \rightarrow \text{fun } y \rightarrow x \text{ ()}$$

$$\text{false} \stackrel{\text{def}}{=} \text{fun } x \rightarrow \text{fun } y \rightarrow y \text{ ()}$$

$$\text{opif} \stackrel{\text{def}}{=} \text{fun } p \rightarrow \text{fst } p \text{ (fst (snd } p)) \text{ (snd (snd } p))}$$

here, () denotes a constant whose value is not significant.

remarks on *opif* and *opfix*

Remark.

- recall that we defined the conditional as

$$\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \stackrel{\text{def}}{=} \text{opif}(e_1, ((\text{fun } _ \rightarrow e_2), (\text{fun } _ \rightarrow e_3)))$$

we can define Boolean constants and the primitive operator *opif* as

$$\begin{aligned} \text{true} & \stackrel{\text{def}}{=} \text{fun } x \rightarrow \text{fun } y \rightarrow x () \\ \text{false} & \stackrel{\text{def}}{=} \text{fun } x \rightarrow \text{fun } y \rightarrow y () \\ \text{opif} & \stackrel{\text{def}}{=} \text{fun } p \rightarrow \text{fst } p (\text{fst } (\text{snd } p)) (\text{snd } (\text{snd } p)) \end{aligned}$$

here, $()$ denotes a constant whose value is not significant.

- similarly, we define the fixed-point operator as

$$\text{opfix} \stackrel{\text{def}}{=} \text{fun } f \rightarrow (\text{fun } x \rightarrow f (\text{fun } y \rightarrow x \times y)) (\text{fun } x \rightarrow f (\text{fun } y \rightarrow x \times y))$$



Interpreter

interpreter

we can code an **interpreter** following the rules of natural semantics

we use a type for the abstract syntax of expressions

```
type expression = ...
```

and we will define a function

```
val eval: expression -> expression
```

corresponding to the relation $\rightarrow$ (because of the determinism of natural semantics)

interpreter for mini-ML

```
type expression =  
  | Var of string  
  | Const of int  
  | Op of string  
  | Fun of string * expression  
  | App of expression * expression  
  | Pair of expression * expression  
  | Let of string * expression * expression
```

interpreter for mini-ML

we have to code the substitution operation $e[x \leftarrow v]$

```
val subst: expression -> string -> expression -> expression
```

we suppose that v is closed.

```
let rec subst e x v =
  match e with
  | Var y -> if y = x then v else e
  | Const _ | Op _ -> e
  | Fun (y, e1) -> if y = x then e else Fun (y, subst e1 x v)
  | App (e1, e2) -> App (subst e1 x v, subst e2 x v)
  | Pair (e1, e2) -> Pair (subst e1 x v, subst e2 x v)
  | Let (y, e1, e2) ->
    Let (y, subst e1 x v, if y = x then e2 else subst e2 x v)
```

interpreter for mini-ML

the natural semantics is realized by the function

```
val eval: expression -> expression
```

```
let rec eval = function
  | Const _ | Op _ | Fun _ as v -> v
  | Pair (e1, e2) -> Pair (eval e1, eval e2)
  | Let(x, e1, e2) -> eval (subst e2 x (eval e1))
  | App (e1, e2) ->
    begin match eval e1 with
      | Fun (x, e) -> eval (subst e x (eval e2))
      | Op "+" ->
        let (Pair (Const n1, Const n2)) = eval e2 in
        Const (n1 + n2)
      | Op "fst" ->
        let (Pair(v1, v2)) = eval e2 in v1
      | Op "snd" ->
        let (Pair(v1, v2)) = eval e2 in v2
    end
```

example of evaluation

```
# eval
(Let
  ("x",
    App (Op "+", Pair (Const 1, Const 20)),
    App (Fun ("y", App (Op "+", Pair (Var "y", Var "y"))),
      Var "x"))));;
```

```
- : expression = Const 42
```

interpreter for mini-ML

the pattern matching is intentionally non-exhaustive

```
# eval (Var "x");;
```

```
# eval (App (Const 1, Const 2));;
```

```
Exception: Match_failure ("", 87, 6).
```

we might prefer an option type, an explicit exception, etc.

interpreter for mini-ML

the evaluation may not terminate

for example

$$(\text{fun } x \rightarrow x \ x) (\text{fun } x \rightarrow x \ x)$$

```
# let b = Fun ("x", App (Var "x", Var "x")) in  
  eval (App (b, b));;
```

Interrupted.

exercise

add the operators *opif* and *opfix* into this interpreter.

little interlude

our MINI-ML interpreter is not very efficient because it spends its time making substitutions and therefore reconstructing expressions

question: can we avoid the substitution operation?

little interlude

our MINI-ML interpreter is not very efficient because it spends its time making substitutions and therefore reconstructing expressions

question: can we avoid the substitution operation?

idea: we interpret the expression e using an **environment** providing the current value of each variable (i.e. a dictionary)

```
val eval: environment -> expression -> value
```

little interlude

our MINI-ML interpreter is not very efficient because it spends its time making substitutions and therefore reconstructing expressions

question: can we avoid the substitution operation?

idea: we interpret the expression e using an **environment** providing the current value of each variable (i.e. a dictionary)

```
val eval: environment -> expression -> value
```

difficulty: the result of

$$\text{let } x = 1 \text{ in fun } y \rightarrow +(x, y)$$

is a function which has to “remember” that $x = 1$.

little interlude

our MINI-ML interpreter is not very efficient because it spends its time making substitutions and therefore reconstructing expressions

question: can we avoid the substitution operation?

idea: we interpret the expression e using an **environment** providing the current value of each variable (i.e. a dictionary)

```
val eval: environment -> expression -> value
```

difficulty: the result of

$$\text{let } x = 1 \text{ in fun } y \rightarrow +(x, y)$$

is a function which has to “remember” that $x = 1$.

solution: we have to use a **closure**

little interlude

we use the module `Map` for the environment

```
module Smap = Map.Make(String)
```

we define a new type for the values

```
type value =  
  | Vconst of int  
  | Vop     of string  
  | Vpair   of value * value  
  | Vfun    of string * environment * expression
```

```
and environment = value Smap.t
```

```
val eval: environment -> expression -> value
```

little interlude

```

let rec eval env = function
  | Const n -> Vconst n
  | Op op -> Vop op
  | Pair (e1, e2) -> Vpair (eval env e1, eval env e2)
  | Var x -> Smap.find x env
  | Let (x, e1, e2) -> eval (Smap.add x (eval env e1) env) e2
  | Fun (x, e) -> Vfun (x, env, e)
  | App (e1, e2) ->
    begin match eval env e1 with
    | Vfun (x, clos, e) -> eval (Smap.add x (eval env e2) clos) e
    | Vop "+" ->
      let Vpair (Vconst n1, Vconst n2) = eval env e2 in
      Vconst (n1 + n2)
    | Vop "fst" -> let Vpair (v1, _) = eval env e2 in v1
    | Vop "snd" -> let Vpair (_, v2) = eval env e2 in v2
    end
end

```

note: this is how we will do when we compile ML in the later lecture

exercise

Add the operator *opif* into this interpreter

note: adding the operator *opfix* is much more complicated

weaknesses of natural semantics

natural semantics makes no distinction between programs that crash, such as

$$1\ 2$$

and programs whose evaluation does not terminate, such as

$$(\text{fun } x \rightarrow x\ x)\ (\text{fun } x \rightarrow x\ x)$$

Small-steps semantics

small-steps operational semantics

small-step operational semantics remedies this by introducing a notion of elementary computation step $e_1 \rightarrow e_2$, which we will iterate

small-steps operational semantics

small-step operational semantics remedies this by introducing a notion of elementary computation step $e_1 \rightarrow e_2$, which we will iterate

then we can distinguish

small-steps operational semantics

small-step operational semantics remedies this by introducing a notion of elementary computation step $e_1 \rightarrow e_2$, which we will iterate

then we can distinguish

1. successful termination to a value

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

small-steps operational semantics

small-step operational semantics remedies this by introducing a notion of elementary computation step $e_1 \rightarrow e_2$, which we will iterate

then we can distinguish

1. successful termination to a value

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

2. evaluation stuck on an irreducible e_n which is not a value

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow e_n$$

small-steps operational semantics

small-step operational semantics remedies this by introducing a notion of elementary computation step $e_1 \rightarrow e_2$, which we will iterate

then we can distinguish

1. successful termination to a value

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

2. evaluation stuck on an irreducible e_n which is not a value

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow e_n$$

3. non-terminating evaluation

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots$$

small-steps operational semantics for mini-ML

we start by defining a simpler relation $\xrightarrow{\epsilon}$, corresponding to a “head” reduction, i.e., involving the most external construction of the expression

two rules:

$$\begin{aligned} (\text{fun } x \rightarrow e) \ v \ &\xrightarrow{\epsilon} e[x \leftarrow v] \\ \text{let } x = v \text{ in } e \ &\xrightarrow{\epsilon} e[x \leftarrow v] \end{aligned}$$

note: these two rules reflect the choice of a strategy of **call by value**, as for semantics in big steps.

small-steps operational semantics for mini-ML

we also have reduction rules in mind for the primitives:

$$+(n_1, n_2) \xrightarrow{\epsilon} n \quad \text{with } n = n_1 + n_2$$

$$\text{fst } (v_1, v_2) \xrightarrow{\epsilon} v_1$$

$$\text{snd } (v_1, v_2) \xrightarrow{\epsilon} v_2$$

$$\text{opfix } (\text{fun } f \rightarrow e) \xrightarrow{\epsilon} e[f \leftarrow \text{opfix } (\text{fun } f \rightarrow e)]$$

$$\text{opif } (\text{true}, ((\text{fun } _ \rightarrow e_1), (\text{fun } _ \rightarrow e_2)))) \xrightarrow{\epsilon} e_1$$

$$\text{opif } (\text{false}, ((\text{fun } _ \rightarrow e_1), (\text{fun } _ \rightarrow e_2)))) \xrightarrow{\epsilon} e_2$$

small-steps operational semantics for mini-ML

to reduce in depth, we introduce the inference rule

$$\frac{e_1 \xrightarrow{\epsilon} e_2}{E(e_1) \rightarrow E(e_2)}$$

where E is a **context**, defined by the following grammar

$$\begin{aligned}
 E &::= \square \\
 &| E \ e \\
 &| \nu \ E \\
 &| \text{let } x = E \text{ in } e \\
 &| (E, e) \\
 &| (\nu, E)
 \end{aligned}$$

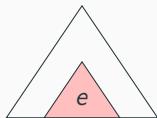
context

a context is a “term with hole” which is represented by the symbol $\square$

Example

$$E \stackrel{\text{def}}{=} \text{let } x = +(2, \square) \text{ in let } y = +(x, x) \text{ in } y$$

$E(e)$ denotes the context E in which $\square$ has been replaced by e



Example

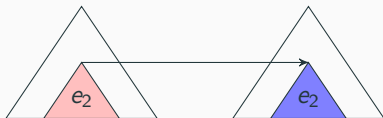
$$E(+(10, 9)) = \text{let } x = +(2, +(10, 9)) \text{ in let } y = +(x, x) \text{ in } y$$

reduction in a context

the simple rule

$$\frac{e_1 \xrightarrow{\epsilon} e_2}{E(e_1) \rightarrow E(e_2)}$$

therefore allows us to evaluate a subexpression



Example

we have the reduction

$$\frac{+(1, 2) \xrightarrow{\epsilon} 3}{\text{let } x = +(1, 2) \text{ in } +(x, x) \rightarrow \text{let } x = 3 \text{ in } +(x, x)}$$

thanks to the context $E \stackrel{\text{def}}{=} \text{let } x = \square \text{ in } +(x, x)$

evaluation order

with such a choice, the contexts here imply an evaluation by call by value and “from left to right”

thus, $(+(1, 2), \square)$ is not an evaluation context

we could have made another choice, such as evaluating “from right to left”, or even not setting an order of evaluation

in the case of `MINI-ML`, this would not make a difference to the value of an expression, but in a more complex language with side effects or exceptional behaviors, evaluating in one order rather than another might make a difference.

definition

Defn. (evaluation and normal form)

We denote by $\rightarrow^*$ the reflective and transitive closure of the relation $\rightarrow$.

(i.e. $e_1 \xrightarrow{*} e_2$ iff e_1 reduces to e_2 by zero or more steps)

We call an expression e **normal form** if there is no expression e' such that $e \rightarrow e'$.



note: obviously, values are normal forms;

normal forms that are not values are bad expressions (e.g. $1\ 2$)

interpreter w.r.t. small-steps semantics

programming the small-steps semantics

we are going to write the following functions:

```
val head_reduction: expression -> expression
```

corresponds to $\xrightarrow{\epsilon}$

```
val decompose: expression -> context * expression
```

decomposes an expression of the form $E(e)$
with e is head reducible

```
val reduce1: expression -> expression option
```

corresponds to $\rightarrow$

```
val reduce: expression -> expression
```

corresponds to $\xrightarrow{*}$

programming the small-steps semantics

we start by characterize the values

```
let rec is_a_value = function
  | Const _ | Op _ | Fun _ ->
    true
  | Var _ | App _ | Let _ ->
    false
  | Pair (e1, e2) ->
    is_a_value e1 && is_a_value e2
```

programming the small-steps semantics

then we write the head reduction

```
let head_reduction = function
  | App (Fun (x, e1), e2) when is_a_value e2 ->
    subst e1 x e2
  | Let (x, e1, e2) when is_a_value e1 ->
    subst e2 x e1
  | App (Op "+", Pair (Const n1, Const n2)) ->
    Const (n1 + n2)
  | App (Op "fst", Pair (e1, e2))
    when is_a_value e1 && is_a_value e2 ->
    e1
  | App (Op "snd", Pair (e1, e2))
    when is_a_value e1 && is_a_value e2 ->
    e2
  | _ ->
    raise NoReduction
```

programming the small-steps semantics

a context E can be represented directly by the function $e \mapsto E(e)$

```
type context = expression -> expression
```

```
let hole = fun e -> e
let app_left ctx e2 = fun e -> App (ctx e, e2)
let app_right v1 ctx = fun e -> App (v1, ctx e)
let pair_left ctx e2 = fun e -> Pair (ctx e, e2)
let pair_right v1 ctx = fun e -> Pair (v1, ctx e)
let let_left x ctx e2 = fun e -> Let (x, ctx e, e2)
```

programming the small-steps semantics

```

let rec decompose e = match e with
(* what we can not decompose *)
  | Var _ | Const _ | Op _ | Fun _ -> raise NoReduction
(* case of one head reduction *)
  | App (Fun (x, e1), e2) when is_a_value e2 -> (hole, e)
  | Let (x, e1, e2) when is_a_value e1 -> (hole, e)
  | App (Op "+", Pair (Const n1, Const n2)) -> (hole, e)
  | App (Op ("fst" | "snd"), Pair (e1, e2))
    when is_a_value e1 && is_a_value e2 ->
    (hole, e)

```

programming the small-steps semantics

```
(* case of in depth reduction *)  
| App (e1, e2) ->  
  if is_a_value e1 then  
    let (ctx, rd) = decompose e2 in  
    (app_right e1 ctx, rd)  
  else  
    let (ctx, rd) = decompose e1 in  
    (app_left ctx e2, rd)  
| Let (x, e1, e2) ->  
  let (ctx, rd) = decompose e1 in  
  (let_left x ctx e2, rd)  
| Pair (e1, e2) -> ...
```

programming the small-steps semantics

```
let reduce1 e =  
  try  
    let ctx, e' = decompose e in  
    Some (ctx (head_reduction e'))  
  with NoReduction ->  
    None
```

finally

```
let rec reduce e =  
  match reduce1 e with None -> e | Some e' -> reduce e'
```

efficiency

such an interpreter is not very efficient

it spent too much time to recalculate the context and then “forget” it

we can do better, for instance, by using a *zipper* [HUE97]

equivalence between two operational semantics

equivalence between two operational semantics

we will show that the two operational semantics, big-steps and small-steps, are equivalent for expressions whose evaluation ends on a value. i.e.

$$e \twoheadrightarrow v \quad \text{if and only if} \quad e \xrightarrow{*} v$$

equivalence between two operational semantics

Lem. (transitions to the context reductions)

Suppose that $e \rightarrow e'$. Then for any expression e_2 and any value v , we have

- $e \ e_2 \rightarrow e' \ e_2$
- $v \ e \rightarrow v \ e'$
- $\text{let } x = e \text{ in } e_2 \rightarrow \text{let } x = e' \text{ in } e_2$

Proof.

From $e \rightarrow e'$ we know that there is an context E such that

$$e = E(r), \quad e' = E(r'), \quad r \xrightarrow{\epsilon} r'$$

Consider the context $E_1 \stackrel{\text{def}}{=} E \ e_2$, then



equivalence between two operational semantics

Lem. (transitions to the context reductions)

Suppose that $e \rightarrow e'$. Then for any expression e_2 and any value v , we have

- $e \ e_2 \rightarrow e' \ e_2$
- $v \ e \rightarrow v \ e'$
- $\text{let } x = e \text{ in } e_2 \rightarrow \text{let } x = e' \text{ in } e_2$

Proof.

From $e \rightarrow e'$ we know that there is an context E such that

$$e = E(r), \quad e' = E(r'), \quad r \xrightarrow{\epsilon} r'$$

Consider the context $E_1 \stackrel{\text{def}}{=} E \ e_2$, then

$$\frac{r \xrightarrow{\epsilon} r'}{E_1(r) \rightarrow E_1(r')} \quad \text{i.e.} \quad \frac{r \xrightarrow{\epsilon} r'}{e \ e_2 \rightarrow e' \ e_2}$$

The other cases can be proceeded similarly.

equivalence between two operational semantics

Prop. (big-steps implies small-steps)

If $e \twoheadrightarrow v$, then $e \xrightarrow{*} v$.



equivalence between two operational semantics

Prop. (big-steps implies small-steps)

If $e \twoheadrightarrow v$, then $e \xrightarrow{*} v$.



Proof.

We proceed by induction on the derivation of $e \twoheadrightarrow v$. Suppose that the last rule is that of a function application:

$$\frac{e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) \quad e_2 \twoheadrightarrow v_2 \quad e_3[x \leftarrow v_2] \twoheadrightarrow v}{e_1 \ e_2 \twoheadrightarrow v}$$

equivalence between two operational semantics

Prop. (big-steps implies small-steps)

If $e \twoheadrightarrow v$, then $e \xrightarrow{*} v$.



Proof.

We proceed by induction on the derivation of $e \twoheadrightarrow v$. Suppose that the last rule is that of a function application:

$$\frac{e_1 \twoheadrightarrow (\text{fun } x \rightarrow e_3) \quad e_2 \twoheadrightarrow v_2 \quad e_3[x \leftarrow v_2] \twoheadrightarrow v}{e_1 \ e_2 \twoheadrightarrow v}$$

By induction hypothesis for each three derivations in the premise and denote by v_1 the value of $\text{fun } x \rightarrow e_3$, we have the three evaluations in small-steps

$$e_1 \rightarrow \cdots \rightarrow v_1$$

$$e_2 \rightarrow \cdots \rightarrow v_2$$

$$e_3[x \leftarrow v_2] \rightarrow \cdots \rightarrow v$$

equivalence between two operational semantics

By passing to the context, we also have the three evaluations

$$e_1 \ e_2 \rightarrow \cdots \rightarrow v_1 \ e_2$$

$$v_1 \ e_2 \rightarrow \cdots \rightarrow v_1 \ v_2$$

$$e_3[x \leftarrow v_2] \rightarrow \cdots \rightarrow v$$

equivalence between two operational semantics

By passing to the context, we also have the three evaluations

$$e_1 \ e_2 \rightarrow \cdots \rightarrow v_1 \ e_2$$

$$v_1 \ e_2 \rightarrow \cdots \rightarrow v_1 \ v_2$$

$$e_3[x \leftarrow v_2] \rightarrow \cdots \rightarrow v$$

By inserting the reduction

$$(\text{fun } x \rightarrow e_3) \ v_2 \xrightarrow{\epsilon} e_3[x \leftarrow v_2]$$

between the second and third lines, we obtain the complete reduction

$$e_1 \ e_2 \rightarrow \cdots \rightarrow v.$$

The other cases are left as exercises. ■

equivalence between two operational semantics

to show the other implication, that is, “small-steps implies big-steps”, we start by establishing two lemmas. The first is obvious.

Lem. (values are already evaluated)

$v \twoheadrightarrow v$ for any value v .



equivalence between two operational semantics

Lem. (reduction and evaluation)

If $e \rightarrow e'$ and $e' \twoheadrightarrow v$, then $e \twoheadrightarrow v$.



equivalence between two operational semantics

Lem. (reduction and evaluation)

If $e \rightarrow e'$ and $e' \twoheadrightarrow v$, then $e \twoheadrightarrow v$.



Proof.

We start with the head reductions, i.e., $e \xrightarrow{\epsilon} e'$. Suppose for example that $e = (\text{fun } x \rightarrow e_1) v_2$ and $e' = e_1[x \leftarrow v_2]$. We can construct the reduction

$$\frac{(\text{fun } x \rightarrow e_1) \twoheadrightarrow (\text{fun } x \rightarrow e_1) \quad v_2 \twoheadrightarrow v_2 \quad e_1[x \leftarrow v_2] \twoheadrightarrow v}{(\text{fun } x \rightarrow e_1) v_2 \twoheadrightarrow v}$$

using the previous lemma ($v_2 \twoheadrightarrow v_2$) and the hypothesis $e' \twoheadrightarrow v$. Other cases of head reduction are treated in a similar manner.

Let us now show that if $e \xrightarrow{\epsilon} e'$ and $E(e') \twoheadrightarrow v$, then $E(e) \twoheadrightarrow v$ for any context E by the structural induction on E (or if one prefers, by induction on the height of the context). We have just done the base case $E = \square$.

Equivalence between two operational semantics

Consider for instance the case $E = E' e_2$. We have $E(e') \rightarrow v$, i.e., $E'(e') e_2 \rightarrow v$. In the case of abstraction, this reduction is of the form

$$\frac{E'(e') \rightarrow (\text{fun } x \rightarrow e_3) \quad e_2 \rightarrow v_2 \quad e_3[x \leftarrow v_2] \rightarrow v}{E'(e') e_2 \rightarrow v}$$

By induction hypothesis, we have $E'(e) \rightarrow (\text{fun } x \rightarrow e_3)$ and hence

$$\frac{E'(e) \rightarrow (\text{fun } x \rightarrow e_3) \quad e_2 \rightarrow v_2 \quad e_3[x \leftarrow v_2] \rightarrow v}{E'(e) e_2 \rightarrow v}$$

that is, $E(e) \rightarrow v$. The other cases are left as exercises. ■

equivalence between two operational semantics

Prop. (small-steps implies big-steps)

If $e \xrightarrow{*} v$ then $e \twoheadrightarrow v$.



Proof.

Let us write the reduction in small steps in the form

$$e \rightarrow e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow e_n \rightarrow v$$

We have $v \twoheadrightarrow v$ and hence by previous lemma $e_n \twoheadrightarrow v$.

Similarly, $e_{n-1} \twoheadrightarrow v$ and so on until $e \twoheadrightarrow v$.



semantics for WHILE language

what about imperative languages?

we can define an operational semantics, in big-steps or in small-steps, for a language with imperative features.

e	$::=$	expression
	c	constant (true, 42, ...)
	x	global variable
	$e \text{ op } e$	binary operation (+, <, ...)
c	$::=$	constant
	n	integer constant (-17, 42, ...)
	b	boolean constant(true, false)

what about imperative languages?

$s ::=$	statement
$x \leftarrow e$	assignment
$\text{if } e \text{ then } s \text{ else } s$	conditional
$\text{while } e \text{ do } s$	loop
$s; s$	sequence
skip	do nothing

what about imperative languages?

typically we associate an **environment/state** S to evaluate/reduce an expression:

$$S, e \twoheadrightarrow S', v \quad \text{or} \quad S, e \rightarrow S', e'$$

and, for instance, define inference rule like

$$\frac{S, e \twoheadrightarrow S', v}{S, x := e \twoheadrightarrow S' \oplus \{x \mapsto v\}, \text{void}}$$

where S can be decomposed into many elements, for modeling a stack, a heap and others.

next

- HW2: a MINI-PYTHON interpreter
 - in OCAML
 - take your time to read and understand the code that is provide
- next: lexical analysis

[illegible]

References i



GÉRARD HUET.

The zipper.

Journal of Functional Programming, 7(5):549–554, 1997.



Benjamin C. Pierce.

Types and Programming Languages.

The MIT Press, 1st edition, 2002.

Questions?